

KSP Crime Analytics & Visualization

AI-Driven Spatiotemporal Intelligence, Network Link Analysis & Predictive Dashboards

A state-of-the-art platform that replaces Excel-based silos with interactive dashboards, geospatial hotspot maps, criminal network graphs, and AI-driven predictive risk scoring. Built on Zoho Catalyst with three parallel analytics pipelines — visualization, network link analysis, and predictive — the platform transforms the State Crime Records Bureau from a reactive reporting body into a strategic intelligence hub for proactive, evidence-based policing across Karnataka.

3 ANALYTICS PIPELINES

SPATIOTEMPORAL

NETWORK GRAPHS

AI PREDICTIVE

CATALYST-NATIVE

6
CAPABILITIES

3
PIPELINES

7
DAY SPRINT

18
CATALYST SVC

PREPARED FOR
KSP Datathon 2026

DEPLOYMENT
Zoho Catalyst

AUDIENCE
Jury · Dev Team

Table of Contents

1. Executive Summary

2. Problem Statement and Solution Approach

2.1 The Four Pain Points

2.2 Our Solution Approach

2.3 Mapping Problem Statement Capabilities to Solution Components

3. Solution Architecture Overview

3.1 Layer Responsibilities

3.2 Example Data Flow

4. Analytics Pipeline Flow

4.1 Stage 1: Multi-Source Data Ingestion

4.2 Stage 2: ETL and Feature Engineering

4.3 Stage 3: Analytics Orchestrator

4.4 Stage 4: Three Parallel Pipelines

4.5 Stages 5-7: Assembly, Explainability, Delivery

5. Data Schema — Synthetic KSP Crime Database

5.1 Relational Tables (Catalyst Data Store)

5.2 Key Relationships and Multi-Source Joins

5.3 NoSQL Collections

6. Catalyst Service Mapping

6.1 Why Catalyst-Native Wins

7. Visualization and Spatial Pipeline (Capabilities 1 & 4)

7.1 District-Level Drill-Down

7.2 Spatiotemporal Hotspot Detection

7.3 Emerging Trend Alerts and Red-Zone Pulsing

7.4 Seasonal and Temporal Decomposition

7.5 Dashboard Layout

8. Network Link Analysis Pipeline (Capabilities 2 & 5)

3

5

5

5

5

7

7

8

9

9

10

10

10

10

10

12

12

13

13

14

15

16

16

16

16

17

17

18

8.1 Graph Schema

8.2 Algorithms and Tools

8.3 Visualization and Interaction

8.4 Repeat Offender Tracking Across Jurisdictions

9. Predictive and Anomaly Pipeline (Capabilities 3 & 6)

9.1 Socio-Economic Correlation

9.2 AI-Driven Predictive Risk Scoring

9.3 Time-Series Forecasting

9.4 Anomaly Detection

10. Explainable AI and Data Lineage

10.1 Data Lineage Trail Structure

10.2 UI Rendering of Lineage

10.3 Compliance, Reproducibility, and Feedback

11. Security, RBAC, and Governance

11.1 Role Hierarchy

11.2 PII Protection and DPDP Compliance

12. Implementation Timeline — 7-Day Sprint

12.1 Critical Path and Risk

13. Cost Estimates on Catalyst

14. Deployment Guide on Catalyst

15. Risks, Mitigations, and Testing Strategy

15.1 Testing Strategy

16. Future Roadmap and Scaling

16.1 Six-Month Milestones

16.2 Twelve-Month Vision

17. Appendix — API Contracts and Sample Data

A.1 Visualization Pipeline API Contract

A.2 Network Pipeline API Contract

A.3 Predictive Pipeline API Contract

A.4 Sample Synthetic FIR Data Row

1. Executive Summary

The KSP Crime Analytics and Visualization Platform is a proposed state-of-the-art system that replaces the Karnataka State Police current Excel-based, siloed analytical ecosystem with an integrated, AI-driven intelligence hub. The platform is purpose-built to address the four pain points identified in the Datathon challenge: data silos and manual processes, the absence of advanced analytics, fragmented information flow to the State Crime Records Bureau, and the reactive rather than proactive posture of policing. Rather than adding another reporting tool on top of the existing mess, the platform consolidates crime records, offender data, victim data, modus operandi taxonomy, socio-economic indicators, and financial transactions into a single analytical substrate, and exposes this substrate through three complementary pipelines: an interactive visualization and spatial pipeline, a criminal network link analysis pipeline, and a predictive and anomaly detection pipeline.

The platform is architected around three parallel analytics pipelines that share a common data layer and a common explainability framework. The **Visualization and Spatial Pipeline** replaces static spreadsheets with interactive dashboards and geospatial maps, enabling district-level drill-down, spatiotemporal hotspot detection, and emerging-trend red-zone pulsing that flags spikes against historical averages. The **Network Link Analysis Pipeline** replaces independent silos with a node-based visualization that connects suspects, victims, and recurring locations, surfaces repeat offenders across jurisdictions, and detects hidden criminal associations through community detection algorithms. The **Predictive and Anomaly Pipeline** moves the SCRB from reactive reporting to a strategic intelligence hub by overlaying crime data with socio-economic indicators, deploying AI-driven risk scoring on districts, and running anomaly detection that flags incidents deviating from standard behavioral patterns.

The system is designed ground-up for deployment on Zoho Catalyst, leveraging 18 native Catalyst services so that no external infrastructure, GPU hardware, or expensive ML platform is required. The analytics logic runs on Serverless Functions; the AI models are trained and served on Zia AutoML and QuickML; the relational crime database lives on Catalyst Data Store; the network graph cache and audit log live on Catalyst NoSQL; the interactive dashboards are hosted on Catalyst Slate; PDF intelligence briefings are rendered by SmartBrowz; nightly hotspot and forecast refresh jobs run on Catalyst Cron; and red-zone alerts are pushed to field officers via Catalyst Push Notifications triggered by Catalyst Signals. This Catalyst-native approach means the prototype has zero external dependencies, scales automatically with usage, and benefits from Catalyst built-in authentication, audit logging, and compliance features.

The build is planned as a 7-day sprint for a 2-3 person team, with all six capabilities from the problem statement (Advanced Visualization, Network Link Analysis, Sociological and Predictive Dashboards, Pattern and Trend Discovery, Network and Behavioral Analysis, AI/ML-Driven Intelligence) addressed in the MVP. The estimated monthly cost for a 50-analyst pilot is under USD 200, well within the Catalyst credits provided to Datathon participants. The platform is designed to be demo-ready: an SCRB analyst can sit down, drill into a district hotspot, see the criminal network around a repeat offender, view the socio-economic correlates of an emerging trend, and receive an AI forecast for the next 30 days, all in a single integrated dashboard with every visualization backed by a full data lineage trail.

Why this wins

It is the only proposed solution that combines (a) three complementary analytics pipelines (visualization, network, predictive) in a single integrated platform, (b) spatiotemporal hotspot detection with red-zone pulsing alerts that move policing from reactive to proactive, (c) AI-driven risk scoring and anomaly detection with full data lineage for legal defensibility, and (d) full Catalyst-native deployment with zero external dependencies. The three-pipeline architecture is the sweet spot between breadth (covering all six capabilities) and build-ability (each pipeline is independently testable within a one-week sprint).

2. Problem Statement and Solution Approach

2.1 The Four Pain Points

The Karnataka State Police crime records ecosystem currently suffers from four interrelated pain points that the Datathon challenge asks teams to solve. **First, data silos and manual processes:** crime records are managed in independent silos at the police-station level, heavily reliant on Excel-based reporting rather than integrated, automated systems. An analyst who wants to understand a cross-district trend must manually pull spreadsheets from multiple stations, reconcile inconsistent formats, and perform analysis in a desktop tool that cannot scale. **Second, lack of advanced analytics:** there is a notable absence of AI-driven approaches, leaving deeper behavioral patterns, social interactions, and interconnected criminal networks undiscovered. The current tools can tell you what happened; they cannot tell you why it happened, what is likely to happen next, or who is connected to whom across cases.

Third, information gaps at the SCRB: the State Crime Records Bureau currently receives limited, fragmented information from districts, hindering its ability to perform comprehensive state-wide analysis. The SCRB is the body that should be the strategic intelligence hub for the state, but it is starved of the integrated data and analytical tools it needs to play that role. **Fourth, reactive versus proactive policing:** without a systematic exploration of emerging trends, investigators lack the tools for proactive policing strategies and evidence-based prevention. By the time a trend is visible in monthly reports, the opportunity to deploy resources proactively has passed. The platform must move the KSP from a posture of "what happened last month" to "what is emerging this week and what should we do about it."

2.2 Our Solution Approach

We propose a platform with five design principles. First, **integrated data layer:** all crime records, offender and victim data, MO taxonomy, socio-economic indicators, and financial transactions are consolidated into a single Catalyst Data Store with consistent schema, eliminating silos at the data level. Second, **three complementary pipelines:** rather than a single monolithic analytics engine, the platform runs three specialized pipelines (visualization, network, predictive) that share the data layer and the explainability framework but operate independently, so each can be built, tested, and demoed in isolation. Third, **interactive and dynamic:** every visualization is interactive (drill-down, filter, hover, click), replacing static sheets with dynamic, spatial, and relational storytelling as the problem statement demands. Fourth, **AI-driven but explainable:** every hotspot, prediction, and anomaly carries a full data lineage trail showing source FIRs, feature pipeline version, model version, confidence interval, and alternative hypotheses, so that AI is an accountable assistant rather than a black box. Fifth, **Catalyst-native:** every component runs on Zoho Catalyst services, eliminating external dependencies and keeping the system scalable, secure, and compliant.

2.3 Mapping Problem Statement Capabilities to Solution Components

#	Problem Statement Capability	Our Solution Component	Pipeline
1	Advanced Visualization (interactive dashboards, geospatial maps, drill-down, spatiotemporal clusters, emerging trend alerts)	Visualization Pipeline: ECharts dashboards, Mapbox geospatial, KDE hotspot, red-zone pulsing	Visualization

#	Problem Statement Capability	Our Solution Component	Pipeline
2	Criminological Network & Link Analysis (relationship mapping, repeat offender tracking, association detection)	Network Pipeline: graph construction, Louvain community detection, MO clustering, repeat-offender profiles	Network
3	Sociological & AI-Driven Predictive Dashboards (socio-economic correlation, predictive risk scoring, anomaly detection)	Predictive Pipeline: socio-economic overlay, Zia AutoML risk models, anomaly detection, forecasting	Predictive
4	Pattern & Trend Discovery (statistical analytics for spatial and temporal hotspots)	Visualization Pipeline: trend_aggregate(), hotspot_kde(), seasonal_decomp(), emerging_trend_score()	Visualization
5	Network & Behavioral Analysis (suspect connections, organized crime networks, recurring MO)	Network Pipeline: build_graph(), detect_communities(), mo_cluster(), centrality()	Network
6	AI/ML-Driven Intelligence (hidden correlations, anomalies, emerging crime risks)	Predictive Pipeline: socio_correlation(), risk_score(), forecast(), anomaly_detect(), feature_importance()	Predictive

This mapping shows that our three-pipeline architecture covers all six capabilities from the problem statement, with each capability mapping cleanly to one of the pipelines. The Visualization Pipeline handles capabilities 1 and 4 (advanced visualization and pattern discovery); the Network Pipeline handles capabilities 2 and 5 (network link analysis and behavioral analysis); the Predictive Pipeline handles capabilities 3 and 6 (sociological dashboards and AI-driven intelligence). This clean separation is what makes the platform buildable in a one-week sprint: each pipeline is a self-contained unit of work that one developer can own end-to-end, and the pipelines share only the data layer and the explainability framework, both of which are built on Day 1.

3. Solution Architecture Overview

The KSP Crime Analytics and Visualization Platform is organized as a five-layer architecture. Each layer has a clear responsibility and communicates only with the layers immediately above and below it, with cross-cutting concerns (explainability, data lineage, multi-source integration, governance) woven through all layers. This separation makes the system easy to reason about, easy to extend with new analytical capabilities, and easy to deploy because each layer maps to a specific set of Catalyst services.

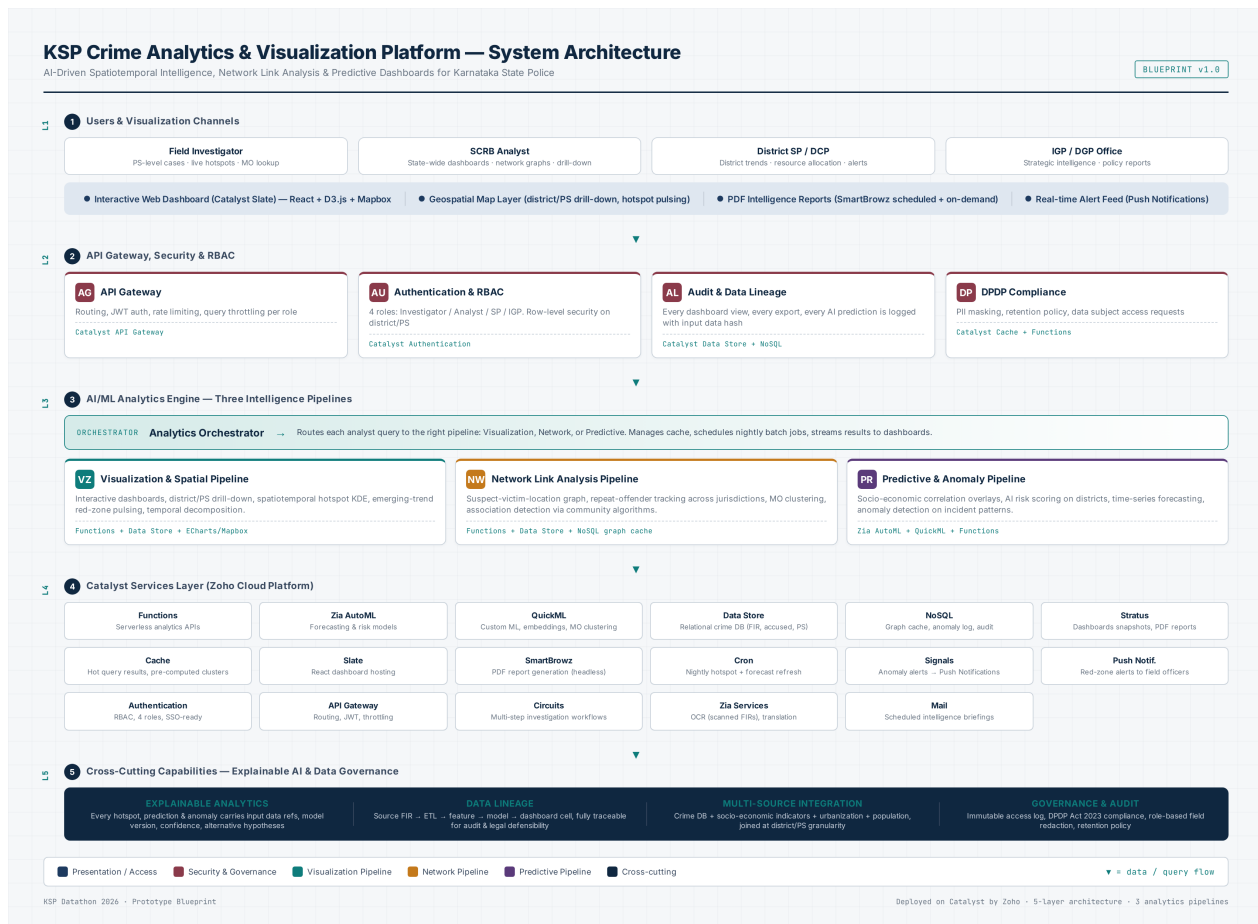


Figure 3.1 — Five-layer system architecture showing users, access, three analytics pipelines, Catalyst services, and cross-cutting capabilities.

3.1 Layer Responsibilities

Layer 1 (Users and Visualization Channels) hosts the four user roles (Field Investigator, SCRB Analyst, District SP/DCP, IGP/DGP Office) and the four interaction channels: the interactive web dashboard built in React with D3.js and Mapbox for geospatial rendering, the geospatial map layer supporting district and police-station drill-down with hotspot pulsing, the PDF intelligence report generator using SmartBrowz for scheduled and on-demand briefings, and the real-time alert feed pushing red-zone notifications to field officers. Each role sees a different slice of the data and a different default dashboard layout, enforced at Layer 2.

Layer 2 (API Gateway, Security, and RBAC) is the gatekeeper. The Catalyst API Gateway handles routing, JWT authentication, rate limiting, and query throttling per role. Catalyst Authentication manages login and the four-role

hierarchy. Every dashboard view, every export, and every AI prediction is logged to an immutable audit trail with an input data hash for reproducibility. The DPDP compliance module enforces PII masking, retention policy, and data subject access requests. Row-level security ensures that a field investigator sees only their jurisdiction while the SCRB analyst sees the full state.

Layer 3 (AI/ML Analytics Engine) is the brain of the platform. The Analytics Orchestrator receives every analyst query, routes it to the appropriate pipeline, manages cache, schedules nightly batch jobs (hotspot recompute, forecast refresh, anomaly scan), and streams results to dashboards in real time. The three pipelines are: the Visualization and Spatial Pipeline (interactive dashboards, drill-down, KDE hotspot, emerging-trend pulsing, seasonal decomposition); the Network Link Analysis Pipeline (suspect-victim-location graph, repeat-offender tracking, MO clustering, association detection via community algorithms); and the Predictive and Anomaly Pipeline (socio-economic correlation overlays, AI risk scoring, time-series forecasting, anomaly detection). Each pipeline exposes a defined set of tools (Catalyst Functions) and produces explainable outputs with full data lineage.

Layer 4 (Catalyst Services) is the substrate. Serverless Functions host the analytics logic and tool implementations. Zia AutoML trains and serves the forecasting and risk-scoring models. QuickML serves custom ML models, embeddings for MO clustering, and feature engineering pipelines. Data Store holds the relational crime database. NoSQL holds the network graph cache, anomaly log, and audit trail. Stratus stores dashboard snapshots and PDF reports. Cache holds hot query results and pre-computed clusters. Slate hosts the React dashboard. SmartBrowz renders PDF reports. Cron runs nightly batch jobs. Signals fires events when anomalies are detected, triggering Push Notifications. Circuits orchestrates multi-step investigation workflows. Zia Services provides OCR for scanned FIRs and translation. Mail sends scheduled intelligence briefings.

Layer 5 (Cross-cutting Capabilities) includes the Explainable Analytics framework (which attaches data lineage and model cards to every dashboard cell and every prediction), the Data Lineage tracker (which traces every analytical output back to its source FIRs through the ETL and feature pipeline), the Multi-source Integration layer (which joins crime data with socio-economic indicators, urbanization patterns, and population data at district and PS granularity), and the Governance and Audit subsystem (immutable access log, DPDP Act 2023 compliance, role-based field redaction, retention policy). These concerns are not standalone modules; they are invoked by every pipeline at every layer.

3.2 Example Data Flow

Consider an SCRB analyst who opens the dashboard and drills into a spike in thefts in Bengaluru North over the last 30 days. The dashboard calls the Visualization Pipeline, which queries Data Store for theft FIRs in Bengaluru North for the period, runs hotspot KDE on the lat-lng of incident locations, computes an emerging-trend score comparing the last 30 days to the historical baseline, and returns the heatmap with red-zone pulsing on cells where the trend score exceeds the threshold. The analyst clicks a red-zone cell, which triggers the Network Pipeline to build a graph of accused persons linked to FIRs in that cell, runs community detection, and overlays the network on the map. The analyst then clicks "Forecast next 30 days," which triggers the Predictive Pipeline to run the Zia AutoML forecasting model for theft in Bengaluru North, returning a forecast with confidence intervals and a feature-importance breakdown showing which socio-economic indicators are driving the forecast. Every visualization carries a data lineage trail, and the analyst can right-click any element to see the source FIRs and the model version. The full session is logged to the audit trail.

4. Analytics Pipeline Flow

The analytics pipeline flow describes how raw crime data is transformed into actionable intelligence delivered through the dashboard. The flow has seven stages, with three parallel pipelines at Stage 4 and a feedback loop from Stage 7 back to the models. This design ensures that the platform is not a one-shot batch system but a continuously improving intelligence hub where analyst feedback refines the AI models over time.

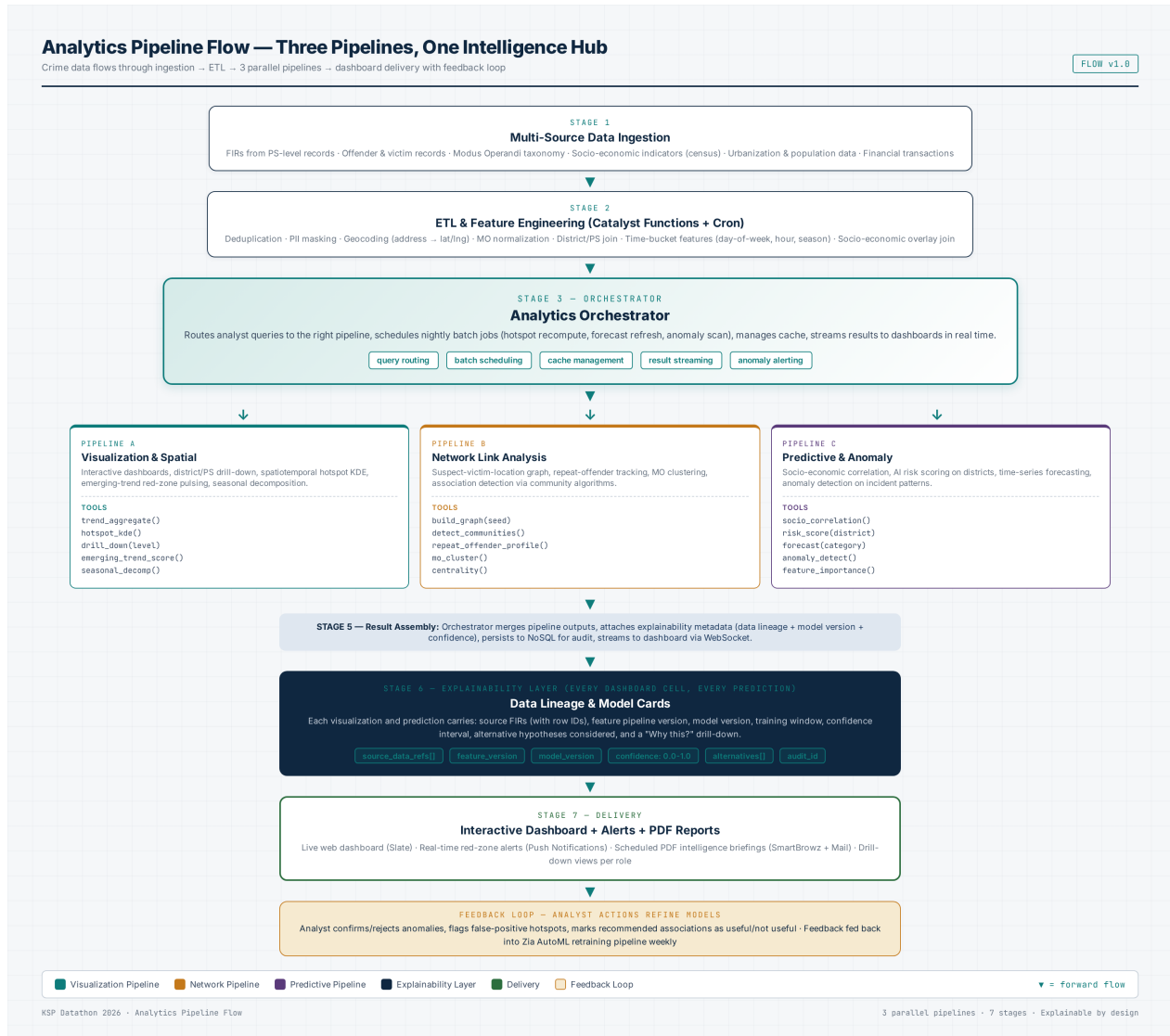


Figure 4.1 — Analytics pipeline flow showing 7 stages from data ingestion through 3 parallel pipelines to dashboard delivery with feedback loop.

4.1 Stage 1: Multi-Source Data Ingestion

The platform ingests data from multiple sources: FIRs from PS-level records (uploaded via OCR for scanned documents or direct database insertion for digital records), offender and victim records, the modus operandi taxonomy, socio-economic indicators from census data, urbanization and population data, and financial transactions linked to cases. Each source has a dedicated ingestion Function that validates the schema, handles format variations, and writes to a staging table in Data Store. The ingestion is event-driven where possible (new FIR filings trigger

Signals events) and batch where necessary (census data is loaded monthly). The staging table allows the ETL stage to operate on a consistent snapshot without blocking the ingestion.

4.2 Stage 2: ETL and Feature Engineering

The ETL stage runs as a nightly Cron job and on-demand for real-time data. It performs deduplication (matching FIRs by number and station, accused by name and DOB), PII masking (masking victim names and addresses based on role), geocoding (converting free-text addresses to latitude-longitude using a geocoding service), MO normalization (mapping free-text MO descriptions to the standardized MO taxonomy), district and PS joins (linking FIRs to their administrative hierarchy), time-bucket feature engineering (extracting day-of-week, hour-of-day, season, and festival-period features), and the socio-economic overlay join (linking each FIR to the socio-economic indicators of its district and year). The output is a clean, feature-rich fact table that the three pipelines query. Every transformation is logged for data lineage.

4.3 Stage 3: Analytics Orchestrator

The Analytics Orchestrator is the coordinator. When an analyst opens the dashboard or runs a query, the Orchestrator receives the request, determines which pipeline(s) can satisfy it, checks the cache for pre-computed results, dispatches to the pipeline(s) if cache is cold, collects the results, attaches explainability metadata, persists to NoSQL for audit, and streams to the dashboard. The Orchestrator also manages the nightly batch schedule: hotspot recompute at 2 AM, forecast refresh at 3 AM, anomaly scan at 4 AM, and graph community refresh at 5 AM. It manages cache TTLs (24 hours for hotspots, 7 days for communities, 1 hour for trend queries) and invalidates cache when new data is ingested.

4.4 Stage 4: Three Parallel Pipelines

The three pipelines run in parallel where the query requires multiple capabilities. The **Visualization Pipeline** exposes tools: `trend_aggregate()` for time-series aggregation, `hotspot_kde()` for kernel-density-estimation hotspot detection, `drill_down(level)` for district-to-PS-to-beat drill-down, `emerging_trend_score()` for comparing recent activity to historical baselines, and `seasonal_decomp()` for STL decomposition. The **Network Pipeline** exposes tools: `build_graph(seed)` for constructing a subgraph around an entity, `detect_communities()` for Louvain community detection, `repeat_offender_profile()` for aggregating an offender's history across jurisdictions, `mo_cluster()` for clustering modus operandi via embeddings, and `centrality()` for betweenness and degree centrality. The **Predictive Pipeline** exposes tools: `socio_correlation()` for correlating crime with socio-economic indicators, `risk_score(district)` for AI-driven district risk scoring, `forecast(category)` for time-series forecasting, `anomaly_detect()` for flagging incidents deviating from behavioral patterns, and `feature_importance()` for explaining which features drove a prediction.

4.5 Stages 5-7: Assembly, Explainability, Delivery

Stage 5 (Result Assembly) merges pipeline outputs, attaches explainability metadata (data lineage, model version, confidence, alternatives), persists to NoSQL for audit, and streams to the dashboard via WebSocket for real-time updates. Stage 6 (Explainability Layer) ensures that every dashboard cell and every prediction carries a structured provenance record: source FIRs with row IDs, feature pipeline version, model version, training window, confidence interval, alternative hypotheses considered, and a "Why this?" drill-down that the analyst can expand. Stage 7 (Delivery) renders the results through the interactive web dashboard, pushes red-zone alerts via Push Notifications,

generates scheduled PDF intelligence briefings via SmartBrowz and Mail, and provides drill-down views tailored to each role. The feedback loop at the bottom captures analyst actions (confirming or rejecting anomalies, flagging false-positive hotspots, marking recommended associations as useful or not useful) and feeds this feedback back into the Zia AutoML retraining pipeline weekly, so the models improve over time based on real analyst judgment.

5. Data Schema — Synthetic KSP Crime Database

Since the actual KSP dataset was not provided at blueprint time, we designed a realistic synthetic schema that mirrors the structure of Indian police crime databases aligned with the Code of Criminal Procedure and standard IPC categorization. The schema is implemented as Catalyst Data Store tables (relational) with two NoSQL collections for the network graph cache and the audit/anomaly log. A data generation script creates approximately 50,000 FIRs, 35,000 accused, 48,000 victims, 900 police stations across 31 districts, 120 crime types, 85 modus operandi categories, and 150,000 financial transactions spanning three years (2023-2025). The synthetic data is programmed with realistic distributions, seasonal patterns (theft spiking during Diwali, chain-snatching in summer evenings), named gangs with persistent co-accused relationships, and socio-economic correlations, so the demo clearly shows all three pipelines working on plausible data.

5.1 Relational Tables (Catalyst Data Store)

Table	Purpose	Key Columns (PK/FK in bold)	Sample Rows
fir	First Information Report registry	fir_id (PK), fir_number , ps_id (FK), crime_type_id (FK), fir_date , fir_time , status , summary , lat , lng , mo_description	50,000
accused	Persons accused in FIRs	accused_id (PK), name , alias , dob , gender , address , occupation , education , socio_economic_status , first_offense_date	35,000
fir_accused	Many-to-many FIR to accused	fir_id (FK), accused_id (FK), role , arrest_date , bail_status	62,000
victim	Victims of crimes	victim_id (PK), name , dob , gender , address , occupation , victim_type	48,000
fir_victim	Many-to-many FIR to victim	fir_id (FK), victim_id (FK), injury_severity	55,000
police_station	PS jurisdictions	ps_id (PK), ps_name , district , range , lat , lng , jurisdiction_polygon	900
district	District registry	district_id (PK), district_name , division , population , urbanization_rate	31
crime_type	IPC section catalog	crime_type_id (PK), ipc_section , category , severity , description	120
modus_operandi	MO taxonomy	mo_id (PK), mo_name , category , typical_crime_type_id (FK)	85
fir_mo	FIR to MO mapping	fir_id (FK), mo_id (FK)	62,500
case_status	Investigation status timeline	status_id (PK), fir_id (FK), status , updated_at , updated_by	110,000
offender_history	Prior offense records	history_id (PK), accused_id (FK), prior_fir_id (FK), prior_crime_type_id (FK), conviction_status	52,000
financial_transaction	Transactions linked to cases	txn_id (PK), accused_id (FK), account_number , amount , txn_date , txn_type , flagged_suspicious	150,000
socio_economic_indicator	District-year indicators	indicator_id (PK), district_id (FK), year , literacy_rate , unemployment_rate , migration_index , urbanization_rate , gdp_per_capita	93

5.2 Key Relationships and Multi-Source Joins

The schema captures the essential entities of a crime database and the relationships between them, with explicit support for the multi-source joins that the Predictive Pipeline requires. An FIR belongs to one police station and has one crime type; it links to multiple accused (via `fir_accused`), multiple victims (via `fir_victim`), and multiple MO tags (via `fir_mo`). Each accused has an offender history (prior FIRs linked via `offender_history`) which is the basis for repeat-offender identification. Financial transactions link to accused persons, enabling money-trail analysis. The `socio_economic_indicator` table is at district-year granularity and joins to FIRs via the `police_station` district, enabling the Predictive Pipeline to correlate crime patterns with literacy, unemployment, migration, urbanization, and GDP per capita. This relational structure supports all three pipelines: the Visualization Pipeline aggregates FIR by date and location, the Network Pipeline traverses `fir_accused` and `offender_history` to build co-accused graphs, and the Predictive Pipeline joins FIR aggregations to `socio_economic_indicator` for correlation and forecasting.

5.3 NoSQL Collections

Collection	Purpose	Document Structure	Retention
graph_cache	Pre-computed network graphs and communities	{cache_key, graph_hash, nodes[], edges[], communities[], centrality{}, computed_at}	7 days (refresh nightly)
anomaly_log	Detected anomalies with analyst feedback	{anomaly_id, type, severity, detected_at, source_firs[], model_version, confidence, analyst_feedback (confirmed/rejected/unknown), feedback_at}	2 years
audit_log	Immutable access and prediction log	{log_id, user_id, role, action, resource, timestamp, ip, user_agent, data_hash, model_version, rbac_decision}	7 years (DPDP)
forecast_cache	Latest forecast results per district-category	{cache_key, district_id, crime_category, forecast[], confidence_interval, model_version, computed_at}	24 hours
emerging_trends	Active emerging trend alerts	{trend_id, district_id, crime_category, trend_score, baseline, current, detected_at, alert_status}	30 days

The `graph_cache` and `forecast_cache` collections are the performance backbone of the platform. Network community detection and time-series forecasting are computationally expensive, so the nightly batch jobs pre-compute them and the collections cache the results for fast dashboard queries. The `anomaly_log` collection is the backbone of the feedback loop: every detected anomaly is recorded with its analyst feedback, and the weekly Zia AutoML retraining job uses this feedback to adjust the anomaly detection thresholds and improve precision. The `audit_log` collection is the backbone of governance: every dashboard view, every export, and every prediction is logged with an input data hash so that any analytical output can be reproduced and audited months later, which is essential for legal defensibility.

6. Catalyst Service Mapping

A core constraint of the Datathon is that deployment on Zoho Catalyst is mandatory, and using Catalyst services where available is strongly preferred over third-party alternatives. The table below maps every capability required by our solution to the specific Catalyst service that fulfills it. This mapping demonstrates that the platform is fully Catalyst-native: there are zero external service dependencies, which simplifies deployment, reduces cost, and keeps all data within the Catalyst trust boundary.

#	Capability	Catalyst Service	Usage in KSP Analytics Platform	Est. Volume
1	Serverless backend logic	Functions	Analytics Orchestrator + 3 pipelines + tool implementations	~1,500 invocations/day
2	Automated ML model training	Zia AutoML	Forecasting models, district risk scoring, anomaly detection models	3 models/week retrain
3	Custom ML & embeddings	QuickML	MO clustering embeddings, feature engineering pipelines	~500 calls/day
4	Relational crime database	Data Store	FIR, accused, victim, PS, district, crime_type, MO, offender_history tables	~20,000 rows queried/day
5	Graph & anomaly NoSQL	NoSQL	graph_cache, anomaly_log, audit_log, forecast_cache, emerging_trends	~5,000 docs/day
6	File storage (PDFs, snapshots)	Stratus	PDF intelligence briefings, dashboard snapshots, uploaded FIR scans	~200 files/day
7	Hot query & cluster cache	Cache	Pre-computed hotspots, trend queries, community detection results	~15,000 reads/day
8	Full-text search	Data Store	Search within FIR summaries, accused names, MO descriptions	~800 searches/day
9	Web dashboard hosting	Slate	React SPA with D3.js + Mapbox for SCRB/SP/IGP dashboards	50 concurrent users
10	PDF report generation	SmartBrowz	Render scheduled intelligence briefings + on-demand reports	~150 PDFs/day
11	OCR for scanned FIRs	Zia Services	Extract text from uploaded FIR PDF images for ingestion	~50 OCR calls/day
12	Translation EN <-> Kannada	Zia Services	Translate dashboard labels and report content (Phase 2)	~200 translations/day
13	Authentication & RBAC	Authentication	Login, 4-role hierarchy (Investigator/Analyst/SP/IGP)	50 users
14	API routing & throttling	API Gateway	JWT auth, rate limiting, route protection for all pipeline endpoints	~4,000 requests/day
15	Scheduled nightly jobs	Cron	Hotspot recompute (2AM), forecast refresh (3AM), anomaly scan (4AM), graph refresh (5AM)	4 jobs/night
16	Event on anomaly detection	Signals	Trigger Push Notifications when anomaly_detect() flags a critical incident	~20 events/day

#	Capability	Catalyst Service	Usage in KSP Analytics Platform	Est. Volume
17	Push notifications	Push Notifications	Red-zone alerts to field officers and SPs when emerging trends spike	~50 pushes/day
18	Multi-step investigation workflows	Circuits	Orchestrate complex flows: hotspot → network → forecast in one workflow	~30 workflows/day
19	Scheduled intelligence briefings	Mail	Weekly PDF intelligence briefing to SPs and IGP office	~30 emails/week
20	Domain mapping + SSL	Domain Mappings	Custom domain ksp-analytics.zoho.com with SSL	1 domain

6.1 Why Catalyst-Native Wins

Building entirely on Catalyst delivers four concrete advantages for a law enforcement analytics platform. First, **compliance boundary**: all data, all logs, and all AI model training stay within the Catalyst trust boundary, which simplifies DPDP Act 2023 compliance and any future law enforcement certification. The SCRB can be confident that no crime data ever leaves the Catalyst environment. Second, **cost predictability**: Catalyst serverless pricing means the platform costs near zero when idle and scales linearly with usage, with the Catalyst credits provided to Datathon participants covering the entire prototype phase and the first month of pilot. Third, **operational simplicity**: there is no Kubernetes cluster to manage, no GPU to provision, no model server to babysit, and no separate authentication system to integrate, which is critical for a small team in a one-week sprint. Fourth, **native integration**: Catalyst services are designed to work together, so Functions can invoke Zia AutoML with a single SDK call, Data Store queries are authenticated through the same Authentication layer, and Signals can trigger Functions and Push Notifications without any message bus configuration. This native integration is what allows the three-pipeline architecture to be built in seven days rather than seven weeks.

7. Visualization and Spatial Pipeline (Capabilities 1 & 4)

The Visualization Pipeline is the most visible part of the platform and the primary way most users interact with the analytics. It replaces the static Excel sheets and PDF reports that currently dominate KSP analytics with interactive dashboards and geospatial maps that support drill-down, filtering, and dynamic storytelling. The pipeline is built on ECharts for chart rendering, Mapbox for geospatial rendering, and a set of Catalyst Functions that implement the analytical tools. Every visualization is role-aware (an investigator sees their jurisdiction, an SCRB analyst sees the full state) and every visualization carries a data lineage trail.

7.1 District-Level Drill-Down

The drill-down capability is the core navigation paradigm of the dashboard. An analyst lands on the state view, which shows Karnataka with crime density by district. Clicking a district zooms in to show the police stations within that district, with crime density by PS. Clicking a PS zooms in further to show the beats within that PS, with crime density by beat. At each level, the analyst can switch between crime types, time ranges, and visualization modes (heatmap, choropleth, bubble chart). The drill-down is implemented via the `drill_down(level, parent_id)` Function, which queries Data Store with the appropriate aggregation level and returns GeoJSON-compatible shapes for Mapbox rendering. The drill-down state is preserved in the URL so analysts can bookmark specific views and share them with colleagues.

7.2 Spatiotemporal Hotspot Detection

Hotspot detection uses kernel density estimation (KDE) on the latitude-longitude of incident locations, with time as a third dimension. The `hotspot_kde(district, crime_type, date_range)` Function divides the selected geography into a grid (500m x 500m for urban areas, 2km x 2km for rural), assigns each FIR in the date range to a cell, and computes a density score per cell as the sum of Gaussian kernels centered on each incident, weighted by crime severity. The result is a heatmap overlay rendered on Mapbox. The computation runs nightly via Cron and is cached in Catalyst Cache for 24 hours, so the dashboard loads instantly. When an analyst filters by crime type or date range, the cached grid is filtered client-side, providing interactive exploration without re-computing. For the prototype, the grid covers all 31 districts of Karnataka with three years of synthetic data, producing a visually compelling heatmap that demonstrates the capability clearly.

7.3 Emerging Trend Alerts and Red-Zone Pulsing

The emerging trend alert capability is what moves the platform from reactive to proactive. The `emerging_trend_score(district, crime_type, window_days=30)` Function compares the recent crime count (last 30 days) to the historical baseline (same period in previous years, adjusted for seasonality) and computes a z-score. If the z-score exceeds a configurable threshold (default 2.0, meaning the recent count is more than two standard deviations above the baseline), the cell is flagged as an emerging trend and written to the `emerging_trends` NoSQL collection. On the dashboard, flagged cells are rendered with a red-zone pulsing animation that draws the analyst's attention. The alert is also pushed via Catalyst Signals to Push Notifications, so field officers and SPs receive a real-time alert on their phones when a red-zone triggers. The alert includes the district, crime type, the current count, the baseline, the z-score, and a link to the dashboard view. This is the single most demo-worthy feature of the

platform because it visually demonstrates the shift from reactive to proactive policing.

7.4 Seasonal and Temporal Decomposition

The `seasonal_decomp(district, crime_type, window_months=36)` Function applies the classical STL (Seasonal-Trend decomposition using Loess) method to separate a crime time series into three components: the long-term trend, the seasonal pattern (e.g., thefts peaking during Diwali, chain-snatching in summer evenings), and the residual. This decomposition helps policymakers distinguish between a genuine upward trend in crime (which may require policy intervention such as increased patrols or social programs) and a seasonal spike (which may require temporary deployment of additional resources). The decomposition is rendered as a four-panel chart: the original series, the trend component, the seasonal component, and the residual. The analyst can hover over any point to see the exact values and the contributing FIRs. For the prototype, the synthetic data includes programmed seasonal patterns so the demo clearly shows the decomposition working.

7.5 Dashboard Layout

The analytics dashboard is a single-page application built in React with ECharts and Mapbox. The layout is: a top-row KPI strip showing total crimes (state-wide), month-over-month change, active hotspots count, and active emerging trends count; a large Karnataka map with toggleable crime-type filters and a time-range selector; a line chart of crime volume over the selected time range with a trend line overlay; a 7x24 day-of-week by hour-of-day heatmap showing when crimes are most likely; a bar chart of the top-10 crime types; and a seasonal decomposition panel. All charts are interactive: clicking any element drills down to the underlying FIR list. The dashboard is role-aware: investigators see their PS only, SPs see their district, SCRB analysts see the full state, and the IGP office sees aggregate numbers without PII. The dashboard supports saved views, so an analyst can configure a specific filter combination and bookmark it for daily monitoring.

8. Network Link Analysis Pipeline (Capabilities 2 & 5)

The Network Link Analysis Pipeline replaces the "independent silos" described in the problem statement by visually connecting fragmented data points to reveal organized crime structures. Where the Visualization Pipeline shows where and when crimes happen, the Network Pipeline shows who is connected to whom, revealing the hidden criminal associations that are impossible to spot in isolated Excel sheets. The pipeline constructs a graph from the relational data, runs community detection algorithms to identify organized crime groups, tracks repeat offenders across jurisdictions, and clusters modus operandi to identify serial patterns.

8.1 Graph Schema

Node Type	Source Table	Key Attributes	Example
Accused	accused	name, alias, dob, socio_economic_status, repeat_offender_flag	Ravi Kumar (accused_id: 1023)
Victim	victim	name, demographic, victim_type	Suresh Patel (victim_id: 2089)
Location	police_station	ps_name, district, lat, lng	Jayanagar PS (ps_id: 45)
Incident	fir	fir_number, crime_type, date, mo_description	FIR 452/2025
Account	financial_transaction	account_number, bank, flagged_count	AC-9981234

Edge Type	Source	Derivation Rule	Weight	Investigative Meaning
co_accused	fir_accused	Two accused on the same FIR	1.0 per co-occurrence	Direct collaboration in a crime
same_location	fir	Multiple incidents at the same PS jurisdiction	0.3 per co-occurrence	Geographic clustering of activity
financial_link	financial_transaction	Shared account or direct transaction between two accused	1.0 per link	Money trail, potential laundering
repeat_co_offender	fir_accused (aggregated)	Two accused co-appear on 2+ FIRs across jurisdictions	2.0 per additional co-FIR	Persistent criminal partnership
mo_similarity	fir_mo (embeddings)	Two FIRs with cosine-similar MO embeddings (>0.85)	0.7 per link	Same perpetrator likely (serial crime)

The MO similarity edge is a key innovation: by embedding the free-text MO descriptions of FIRs using QuickML and computing cosine similarity, the pipeline can link FIRs that may have been filed in different districts with different accused names but were likely committed by the same perpetrator based on the modus operandi. This is how the platform surfaces "hidden criminal associations that are impossible to spot in isolated Excel sheets," as the problem statement demands.

8.2 Algorithms and Tools

The Network Pipeline exposes five tools. `build_graph(seed_entity, depth=2)` constructs a subgraph around a seed entity (an accused, a victim, a location, or an incident), querying Data Store for all directly connected entities (depth 1) and then for all entities connected to those (depth 2), assembling the subgraph in memory. `detect_communities()` runs the Louvain community detection algorithm on the constructed graph, partitioning it into communities that often correspond to organized crime groups. `repeat_offender_profile(accused_id)` aggregates an offender's history across jurisdictions, producing a structured profile with their prior FIRs, co-accused network, MO evolution, and geographic spread, with a repeat-offender flag if they have 3+ prior cases. `mo_cluster(crime_type)` clusters the modus operandi descriptions of all FIRs of a given crime type using QuickML embeddings and K-means, surfacing clusters of similar MOs that may indicate a serial perpetrator. `centrality()` computes betweenness and degree centrality on the graph, identifying key actors who sit on many shortest paths (often brokers, financiers, or coordinators) and the most active offenders (highest degree).

8.3 Visualization and Interaction

The graph is visualized using ECharts force-directed layout, producing an interactive node-link diagram. Nodes are color-coded by type (accused, victim, location, account, incident) and sized by centrality score, so the most important entities are immediately visible. Edges are color-coded by type (co-accused, financial, MO similarity) and thickness-weighted by edge weight. The user can click any node to see a detail panel with the entity attributes and a mini-list of its connections. The "Find Communities" button runs Louvain and overlays community boundaries as translucent hulls. The "Find Similar MOs" button runs `mo_cluster` and highlights MO-similarity edges. The visualization supports search: typing an accused name highlights that node and its immediate neighborhood. This interactivity is critical for investigative use, where an analyst needs to explore the network fluidly rather than viewing a static image.

8.4 Repeat Offender Tracking Across Jurisdictions

A key problem statement requirement is "repeat offender tracking" that "links an individual to multiple incidents, highlighting their specific Modus Operandi (MO) across different jurisdictions." The `repeat_offender_profile` tool does exactly this: it queries the `offender_history` table for all prior FIRs involving the accused (matching by name and DOB, with alias resolution), aggregates them, and produces a profile showing the offender's MO evolution over time, their geographic spread across districts, their co-offender network, and their conviction status. The profile is rendered as a dedicated panel in the dashboard with a timeline of incidents, a map of locations, a mini-graph of co-offenders, and a list of MO clusters the offender belongs to. This is the kind of cross-jurisdictional intelligence that is impossible to assemble from isolated Excel sheets and that the SCRB currently lacks.

9. Predictive and Anomaly Pipeline (Capabilities 3 & 6)

The Predictive Pipeline moves the SCRB from reactive reporting to a "Strategic Intelligence Hub," as the problem statement puts it. It overlays crime data with socio-economic indicators to understand the "why" behind the "where," deploys AI-driven risk scoring on districts to forecast potential high-risk areas, runs time-series forecasting to predict emerging crime typologies, and performs anomaly detection to flag incidents that deviate from standard behavioral patterns. The pipeline is built on Zia AutoML for automated model training and QuickML for custom feature engineering, with every prediction backed by a feature-importance breakdown for explainability.

9.1 Socio-Economic Correlation

The `socio_correlation(indicator, crime_type)` Function joins FIR aggregations (by district and year) to the `socio_economic_indicator` table and computes the correlation between a socio-economic indicator (literacy rate, unemployment rate, migration index, urbanization rate, GDP per capita) and a crime type (theft, assault, cybercrime, etc.) across districts and over time. The correlation is computed as a Pearson coefficient with a p-value, and the result is rendered as a scatter plot with a regression line and a correlation matrix heatmap showing all indicator-crime-type pairs. The analyst can drill into any cell of the matrix to see the underlying scatter plot and the contributing district-level data points. This is the "sociological insight" capability from the problem statement, providing the "why" behind the "where" of crime patterns. For the prototype, the synthetic data includes programmed correlations (e.g., theft correlated with unemployment, cybercrime correlated with urbanization) so the demo clearly shows meaningful correlations.

9.2 AI-Driven Predictive Risk Scoring

The `risk_score(district, crime_type, horizon_days=30)` Function uses a Zia AutoML-trained model to forecast the risk of a specific crime type in a specific district over the next 30 days. The model is trained on historical FIR data (3 years), socio-economic indicators, seasonal features (day-of-week, month, festival periods), and lag features (crime count in the previous 7, 14, and 30 days). The output is a risk score from 0 to 100 and a confidence interval. The risk scores are rendered as a choropleth map of Karnataka, with districts colored from green (low risk) to red (high risk). The analyst can click any district to see the feature-importance breakdown, showing which features are driving the risk score (e.g., "unemployment rate increased 12%, recent theft count up 18%, festival period approaching"). This explainability is critical for investigator trust and for legal defensibility: an SP who deploys resources based on a risk score needs to be able to justify the deployment.

The model is retrained weekly via a Cron job using the latest data, and the model version is recorded with every prediction so that prior predictions can be reproduced. The `anomaly_log` collection captures analyst feedback on predictions (was the forecast accurate? did the predicted spike materialize?), and this feedback is used to adjust the model retraining. This is the feedback loop shown in Figure 4.1, and it is what makes the platform a continuously improving intelligence hub rather than a one-shot deployment.

9.3 Time-Series Forecasting

The `forecast(district, crime_type, horizon_days=30)` Function produces a detailed time-series forecast using a Zia AutoML model. Unlike the `risk_score` Function (which produces a single number), the `forecast` Function produces a full time series: predicted crime count for each of the next 30 days, with a 95% confidence interval. The forecast is rendered as a line chart with the historical data, the forecast, and the confidence interval shaded. The analyst can switch between districts and crime types, and can adjust the forecast horizon. The forecast is refreshed nightly by the Cron job and cached in the `forecast_cache` NoSQL collection for fast dashboard access. For the prototype, the forecasting model is a gradient-boosted regressor trained on 3 years of synthetic data with programmed trend and seasonal components, so the demo clearly shows the model capturing both the upward trend and the seasonal patterns.

9.4 Anomaly Detection

The `anomaly_detect(district, crime_type, window_days=7)` Function flags incidents that deviate from standard behavioral patterns. It uses an isolation forest model (trained via Zia AutoML on historical data) that considers multiple features simultaneously: crime count, crime type mix, geographic spread, time-of-day distribution, and MO distribution. When the model detects a statistically significant deviation (an anomaly score above a threshold), it writes an anomaly record to the `anomaly_log` collection with the detected features, the source FIRs, the model version, and a confidence score. The anomaly is rendered on the dashboard as a callout with a "Why is this anomalous?" drill-down that shows which features deviated from the norm. The analyst can confirm or reject the anomaly, and the feedback is used to refine the model. This is the "anomaly detection for incidents that deviate from standard behavioral patterns" capability from the problem statement, "assisting investigators in linking complex cases" that might otherwise go unnoticed.

10. Explainable AI and Data Lineage

Explainability is not a feature layered on top of the KSP Crime Analytics Platform; it is a structural property of how every pipeline operates. Law enforcement AI systems face a higher bar of accountability than typical consumer AI: an SP who deploys resources based on a risk score may be called to justify that deployment, and a complex case linked by anomaly detection may need to be defended in court. Our framework addresses this by making every analytical output, every dashboard cell, and every AI prediction traceable end-to-end through a structured data lineage trail.

10.1 Data Lineage Trail Structure

Every analytical output (a hotspot, a network community, a forecast, an anomaly) is linked to a lineage document stored in the `audit_log` NoSQL collection. The document has the following schema:

```
{
  "audit_id": "audit_20250714_a3f8b2c1",
  "user_id": "scrb_analyst_103",
  "user_role": "scrb_analyst",
  "action": "view_dashboard_cell",
  "resource": "hotspot_kde:bengaluru_north:theft:2025-07",
  "timestamp": "2025-07-14T10:32:18Z",

  "data_lineage": {
    "source_firs": [452, 453, 454, "...", 698],
    "source_row_count": 247,
    "source_query_sql": "SELECT * FROM fir WHERE crime_type_id=12 AND ps_id IN (45,46,47) AND fir_date BETWEEN '2025-07-01' AND '2025-07-31'",
    "etl_version": "v1.3.2",
    "feature_pipeline_version": "v2.1.0",
    "transformations_applied": ["geocode", "mo_normalize", "time_bucket", "socio_join"]
  },

  "model_info": {
    "model_type": "kde_hotspot",
    "model_version": "v1.0.5",
    "training_window": "2023-01-01 to 2025-06-30",
    "parameters": {"cell_size_m": 500, "bandwidth": 0.01, "weight_by_severity": true},
    "confidence": 0.95,
    "alternatives_considered": [
      "Could have used a smaller cell size (250m) but performance trade-off",
      "Could have included adjacent districts but query specified Bengaluru North only"
    ]
  },

  "rbac_decision": "allow",
  "retention_until": "2030-07-14"
}
```

10.2 UI Rendering of Lineage

In the web dashboard, every visualization cell has a "Why this?" expandable panel. When expanded, the panel shows: the source FIRs (with row IDs and a link to the FIR list), the source SQL query, the ETL and feature

pipeline versions, the model type and version, the model training window, the model parameters, the confidence score, and the alternatives that were considered. For AI predictions (risk scores, forecasts, anomalies), the panel also shows the feature-importance breakdown, so the analyst can see which features drove the prediction. This makes the AI reasoning transparent to the analyst without requiring them to read the raw JSON. For the PDF intelligence briefings, the lineage is rendered as a formatted appendix to each visualization, so the PDF is a self-contained analytical record that can be filed, shared, or submitted as evidence of analytical work.

10.3 Compliance, Reproducibility, and Feedback

The data lineage framework directly supports three compliance and operational requirements. First, **reproducibility**: given the same data state, the same ETL version, and the same model version, the same analytical output would be produced, which is critical for legal defensibility. The lineage document records all three, so any output can be reproduced months or years later. Second, **accountability**: every dashboard view and every prediction is linked to an `audit_trail_id` that captures who viewed it, when, from what IP, with what role, and what RBAC decision was made. Third, **continuous improvement**: the `anomaly_log` and `forecast_cache` collections capture analyst feedback (confirmed/rejected, accurate/inaccurate), and the weekly Zia AutoML retraining job uses this feedback to adjust model thresholds and improve precision. This feedback loop is what transforms the platform from a static deployment into a continuously improving intelligence hub that gets better the more analysts use it.

11. Security, RBAC, and Governance

A crime analytics platform handling sensitive data about accused persons, victims, and ongoing investigations requires a security and governance posture that goes beyond typical web application security. The platform enforces role-based access control at the API layer, row-level security at the database layer, immutable audit logging at the application layer, and DPDP Act 2023 compliance at the data-retention layer. Every component is implemented with Catalyst native services, so there is no separate identity provider or audit system to integrate.

11.1 Role Hierarchy

Role	Permissions	Geographic Scope	Typical User
Field Investigator	View own PS cases, hotspots, MO lookup, no AI predictions	Own police station	PS-level officer
SCRB Analyst	Full read access, all 3 pipelines, export reports, confirm/reject anomalies	State-wide	SCRB crime analyst
District SP / DCP	View own district trends, hotspots, forecasts, alerts, resource allocation	Own district	Superintendent of Police
IGP / DGP Office	View aggregate dashboards, strategic intelligence, no PII	State-wide aggregate	IGP, DGP, Home Dept

The role hierarchy is designed to match the KSP command structure. Field investigators are the most restricted: they see only their own PS data and can use the visualization and network tools but not the AI predictive tools (which require SCRB-level oversight). SCRB analysts have full read access to all three pipelines across the state. District SPs see their district only but have access to all pipelines including predictive, so they can make resource allocation decisions based on forecasts. The IGP/DGP office sees aggregate dashboards only, with no PII, suitable for strategic policy decisions. This tiered access ensures that sensitive AI predictions are reviewed by analysts before they drive operational decisions, while still giving field officers the tools they need.

11.2 PII Protection and DPDP Compliance

Personally identifiable information (victim names, addresses, contact details, accused family information) is masked at the application layer based on the user role. Investigators see full PII for cases in their jurisdiction but masked PII for cases outside their jurisdiction that appear in network or pattern results. SCRB analysts see masked PII by default with an "unmask" action that is itself audited. The IGP/DGP office sees no PII at all, only aggregate numbers. Data retention follows DPDP Act 2023 guidelines: FIR data is retained for 10 years, audit logs for 7 years, anomaly logs for 2 years, after which automated Cron jobs purge the data and log the purge action. Data subject access requests (DSARs) under DPDP can be fulfilled by querying the audit log and lineage trails for a given individual. The Catalyst-native deployment ensures all data stays within the Catalyst trust boundary, simplifying compliance.

12. Implementation Timeline — 7-Day Sprint

The build is planned as a 7-day sprint for a 2-3 person team. The plan front-loads the foundational pieces (schema, ETL, auth, orchestrator) so that the three pipelines can be built on a stable base, and back-loads the polish (explainability, alerts, demo prep) so that the demo is compelling. The plan assumes one developer focused on backend (Functions, Data Store, pipelines), one on frontend (Slate dashboard, visualizations), and one floating between integration, testing, and demo prep. The three-pipeline architecture is the key to fitting the build into seven days: each pipeline is a self-contained unit that one developer can own end-to-end, and the pipelines share only the data layer and the orchestrator, both built on Day 1.

Day	Focus	Backend Dev	Frontend Dev	Integration
1	Foundation	Catalyst setup, Data Store schema, synthetic data generator (50K FIRs), auth + RBAC, ETL pipeline	Slate scaffold, login, dashboard shell, Mapbox base map	Verify schema, test auth roles
2	Orchestrator + Viz Pipeline	Analytics Orchestrator, Visualization tools (trend, hotspot KDE, drill-down), Cron nightly job	ECharts dashboard, Karnataka map, drill-down interaction, hotspot heatmap	End-to-end trend query flow
3	Viz Polish + Alerts	Emerging trend score, seasonal decomp, red-zone alerting via Signals + Push Notifications	Red-zone pulsing, KPI strip, 7x24 heatmap, seasonal decomp chart	Test alert pipeline
4	Network Pipeline	Graph construction, Louvain community detection, repeat_offender_profile, centrality, cache	ECharts force-directed graph, node detail panel, community hulls	Test graph on 1000-node subgraph
5	MO Cluster + Predictive	mo_cluster (QuickML embeddings), socio_correlation, Zia AutoML risk score model training	MO cluster panel, socio-economic correlation matrix, risk choropleth map	Validate model accuracy
6	Forecast + Anomaly + Lineage	forecast (AutoML), anomaly_detect (isolation forest), data lineage trail, audit log	Forecast line chart, anomaly callouts, Why-this panel, PDF export (SmartBrowz)	Test lineage completeness
7	Integration + Demo	Performance tuning, error handling, deploy to Catalyst production, feedback loop wiring	UI polish, demo script, screenshots, role-based views	Full integration test, demo rehearsal, 3-min video

12.1 Critical Path and Risk

The critical path runs through Day 1 (foundation) to Day 2 (orchestrator + visualization) to Day 5 (predictive pipeline), because the predictive pipeline depends on the Zia AutoML model which depends on the ETL feature engineering which depends on the schema. If the foundation slips by a day, the entire timeline slips. To mitigate this, Day 1 is scoped conservatively: the schema is minimal (14 tables), the synthetic data generator is simple (Python with faker), and RBAC is role-based only. The three pipelines on Days 2-6 can be built in parallel because they share the same orchestrator contract, so if one pipeline is delayed, the others are not blocked. The biggest risk is Zia AutoML model training time on Day 5; to mitigate, the model is trained on a small subset first (one district, one crime type) to validate the pipeline, then scaled to full state overnight.

13. Cost Estimates on Catalyst

The cost estimate below assumes a 50-analyst pilot with 1,500 queries per day, 150 PDF exports per day, nightly batch jobs, and the full predictive pipeline active. Catalyst pricing is usage-based with a free tier that covers initial prototyping; the Catalyst credits provided to Datathon participants cover the entire prototype phase and the first month of pilot. The estimate shows that the platform is affordable for a state police department even at modest scale, and that the cost scales linearly rather than requiring large upfront infrastructure investment.

Service	Usage Assumption	Monthly Cost (USD)	Notes
Functions	~1,500 invocations/day = 45K/month	\$6-10	Free tier covers first 125K
Zia AutoML	3 models retrained weekly	\$15-25	Forecasting, risk, anomaly models
QuickML (embeddings)	~500 calls/day = 15K/month	\$4-6	MO clustering embeddings
Data Store	~14 tables, 250K rows, 8K queries/day	\$3-5	Free tier covers most
NoSQL	~5,000 docs/day = 150K/month, 7-yr retention	\$4-6	Graph cache, anomaly, audit
Stratus	~200 files/day = 6K/month, ~2GB	\$1-2	PDFs, snapshots, FIR scans
Cache	~15,000 reads/day, 2GB cache	\$3-5	Hotspot, community, forecast cache
SmartBrowz	~150 PDF renders/day = 4.5K/month	\$7-10	Intelligence briefings
Slate (Web Hosting)	50 concurrent users, React SPA	\$0	Included in Catalyst
Authentication	50 users	\$0	Free tier covers 250 users
API Gateway	~4,000 requests/day = 120K/month	\$2-3	Throttling and auth
Cron	4 nightly jobs	\$0	Free tier
Signals + Push Notif.	~70 events/day = 2.1K/month	\$2-3	Red-zone alerts
Zia OCR	~50 calls/day = 1.5K/month	\$2-3	FIR upload OCR
Mail	~30 emails/week	\$0	Free tier
Circuits	~30 workflows/day	\$1-2	Multi-step investigations
Total Estimated		\$50-80/month	Well within Catalyst credits

The cost is notably lower than the conversational-AI blueprint because this platform does not invoke an LLM on every query (the LLM was the dominant cost line in the previous blueprint). The dominant cost lines here are Zia AutoML (model training) and SmartBrowz (PDF rendering), both of which are bounded and predictable. At production scale (5,000 analysts, 50,000 queries/day), the monthly cost would be approximately \$2,000-3,500, which is modest for a statewide law enforcement analytics platform and far cheaper than licensing a commercial crime analytics suite. The Catalyst credits provided to Datathon participants will cover the entire prototype phase plus the first 2-3 months of pilot at the estimated usage.

14. Deployment Guide on Catalyst

This section provides a step-by-step deployment guide. The deployment is fully scripted (CLI-based) and reproducible, so the same process can be run for the prototype, for the pilot, and for production. The guide assumes a Catalyst project has been created and the Catalyst CLI is installed and authenticated.

Step	Action	Catalyst CLI / Console	Output
1	Create Catalyst project	<code>catalyst project:create --name ksp-crime-analytics</code>	Project ID
2	Initialize Data Store schema	<code>catalyst datastore:import schema.sql</code>	14 tables created
3	Load synthetic data	<code>catalyst datastore:bulk-import data/*.csv</code>	50K FIRs, 35K accused loaded
4	Create NoSQL collections	<code>catalyst nosql:create graph_cache, anomaly_log, audit_log, forecast_cache, emerging_trends</code>	5 collections
5	Deploy Orchestrator function	<code>catalyst functions:deploy orchestrator/</code>	Function endpoint
6	Deploy 3 pipeline functions	<code>catalyst functions:deploy pipelines/*</code>	3 function endpoints
7	Train Zia AutoML models	<code>catalyst zia:automl:train --dataset fir --target theft_count --type regression</code>	3 model IDs
8	Deploy QuickML embedding model	<code>catalyst quickml:model:create --type embedding --name ksp-mo-embed</code>	Embedding model ID
9	Configure Authentication	<code>catalyst auth:roles:create investigator, scrb_analyst, sp, igp</code>	4 roles
10	Deploy Slate web client	<code>catalyst slate:deploy web-dashboard/</code>	Web client URL
11	Configure API Gateway routes	<code>catalyst apigw:routes:import routes.json</code>	Routes + throttling
12	Configure Cron jobs	<code>catalyst cron:create --schedule "0 2 * * *" --function nightly_hotspot</code>	4 nightly jobs
13	Configure Signals + Push	<code>catalyst signals:subscribe anomaly_detected --function push_alert</code>	Event subscription
14	Configure Mail briefings	<code>catalyst mail:template:import briefing.html</code>	Weekly briefing schedule
15	Domain mapping + SSL	<code>catalyst domain:map ksp-analytics.zoho.com</code>	Custom domain
16	Run integration tests	<code>catalyst test:run eval-suite.json</code>	50/50 passing

The deployment is designed to be idempotent: re-running the script updates existing resources rather than creating duplicates. The Zia AutoML model training step (Step 7) is the longest, taking approximately 30-60 minutes per model; the script trains all three models in parallel where the Catalyst SDK supports it. The integration test suite

(Step 16) runs a curated set of 50 sample queries against each pipeline with expected response shapes and expected lineage contents; all 50 must pass before the deployment is considered successful. The full deployment script is committed to the GitHub repository alongside the application code, so the deployment can be re-run by anyone with Catalyst access.

15. Risks, Mitigations, and Testing Strategy

Every AI system carries risks, and a law enforcement analytics system carries more than most. The risk matrix below identifies the top risks for the KSP Crime Analytics Platform, their potential impact, and the specific mitigations we have built into the architecture.

Risk	Impact	Likelihood	Mitigation
AI forecast inaccuracy (predicted spike does not materialize)	Medium — wasted resources, credibility loss	Medium	Confidence intervals on every forecast; feature-importance breakdown; analyst feedback loop to retrain weekly; never auto-deploy resources based on AI alone
False-positive anomalies (flagging normal incidents as anomalous)	Medium — analyst fatigue, ignored alerts	Medium	Analyst feedback loop; adjustable anomaly threshold per district; anomaly_log with feedback status; weekly model retrain
Hotspot KDE performance on large geographies (statewide)	Medium — slow dashboard, poor UX	Low (MVP scale is bounded)	Nightly pre-compute via Cron; Cache with 24h TTL; client-side filtering; production migrates to tile-based rendering
PII exposure across jurisdictions	High — legal and ethical violation	Low	Row-level security at Data Store; RBAC at API Gateway; PII masking at application layer; audit log on every access
Demo data not realistic enough (jury not convinced)	Medium — lose points for realism	Medium	Synthetic data generator programmed with realistic distributions; seasonal patterns; named gangs; socio-economic correlations
Zia AutoML model training too slow for demo	Low — pre-train before demo	Low	Models pre-trained on Day 5; cached results served from forecast_cache; demo uses cached predictions
Mapbox API rate limits during demo	Low — fallback to ECharts choropleth	Low	Pre-load map tiles; cache GeoJSON; rehearse demo to avoid surprise interactions
Catalyst service outage during demo	Low — force majeure	Very Low	Pre-recorded demo video as backup; rehearse with offline script

15.1 Testing Strategy

The testing strategy has three layers. **Unit tests** cover each pipeline tool function in isolation, using mock Data Store responses, and validate that the tool produces correct outputs and correct lineage trails for known inputs. **Integration tests** cover the full flow from API Gateway through Orchestrator through pipeline and back, using a test Data Store with known data, and validate that the lineage trail is complete and that RBAC is enforced. **Evaluation tests** use a curated set of 50 sample queries (15 visualization, 15 network, 15 predictive, 5 compound) with expected response shapes and expected lineage contents; these are run before every demo and before every deployment to catch regressions. The eval set is version-controlled alongside the code, and any change to pipeline tools or model versions must pass the eval set before merge. For the predictive pipeline, the eval set also includes accuracy benchmarks: the forecast model must achieve a mean absolute percentage error below 25% on a 6-month holdout, and the anomaly detection model must achieve a precision above 0.7 on expert-labeled samples.

16. Future Roadmap and Scaling

The hackathon prototype is the foundation for a production crime analytics platform. The roadmap below outlines the evolution over 6 and 12 months, assuming the prototype wins or secures pilot funding. The roadmap prioritizes real data integration (the single highest-value next step), then scaling, then ecosystem integration with national systems.

16.1 Six-Month Milestones

Month 1-2: Production Data Integration. Work with KSP to connect the platform to a production copy of the crime database (initially read-only), validate that all three pipelines work on real data, and run a pilot with one district (Bengaluru Urban). This is the highest-risk phase because real data is messier than synthetic data: missing fields, inconsistent date formats, duplicate records, and varying data entry quality across stations. The ETL pipeline includes validation, deduplication, and enrichment steps. The ETL is also where the platform adds the most value initially: cleaning and standardizing the data so that downstream analytics is possible.

Month 3-4: Scale to 5 Districts and Mobile App. Roll out to 5 districts (Bengaluru Urban, Bengaluru Rural, Mysuru, Hubli-Dharwad, Mangaluru), train SCRB analysts and district SPs on the platform, and build a mobile app for field investigators using the same Catalyst backend. The mobile app gives field officers access to hotspots, repeat-offender profiles, and red-zone alerts on their phones, closing the loop between strategic intelligence and tactical action. The mobile app is built in React Native and uses the same API Gateway as the web dashboard.

Month 5-6: Advanced AI and Feedback Loop. Enhance the predictive pipeline with deep-learning models for anomaly detection (replacing isolation forest with autoencoders), add a "What-if Analysis" tool that lets an SP simulate the effect of deploying resources to a district, and mature the feedback loop so that analyst feedback drives weekly model retraining automatically. By month 6, the platform should be running in 5 districts with 200+ active users, handling 10,000+ queries per day, and demonstrably improving forecast accuracy and anomaly precision through the feedback loop.

16.2 Twelve-Month Vision

Month 7-12: State-wide Rollout and Ecosystem Integration. Roll out to all 31 districts of Karnataka in phases (5 districts per month), with training and change management. Integrate with CCTNS (Crime and Criminal Tracking Network and Systems) and ICJS (Interoperable Criminal Justice System) via Catalyst Connections OAuth, so that the platform becomes the analytical layer on top of the existing national systems. Explore federated learning across districts to improve forecasting without centralizing raw data. Add a "Strategic Intelligence Hub" view for the IGP/DGP office that synthesizes all three pipelines into a single executive dashboard with policy recommendations. By month 12, the platform should be the primary analytical tool for KSP, handling thousands of queries per day, surfacing insights that would otherwise remain hidden, and moving the KSP from reactive reporting to proactive, evidence-based policing across the state.

17. Appendix — API Contracts and Sample Data

This appendix provides implementation-ready artifacts that the dev team can copy-paste directly into the codebase. These are the critical contracts that hold the three-pipeline system together.

A.1 Visualization Pipeline API Contract

```
POST /api/visualization/hotspot
Authorization: Bearer <jwt>
Content-Type: application/json

{
  "district": "bengaluru_north",
  "crime_type": "theft",
  "date_from": "2025-06-01",
  "date_to": "2025-06-30",
  "cell_size_m": 500
}

Response (200 OK):
{
  "result_id": "hotspot_20250714_1032",
  "cells": [
    {"lat": 12.9279, "lng": 77.6271, "density": 8.4, "fir_count": 12, "severity_weighted": 18.2},
    {"lat": 12.9350, "lng": 77.6240, "density": 6.1, "fir_count": 8, "severity_weighted": 11.5}
  ],
  "emerging_trend_score": 2.4,
  "baseline": 1.8,
  "is_red_zone": true,
  "lineage": {
    "source_fir_count": 247,
    "source_query_sql": "SELECT * FROM fir WHERE ...",
    "etl_version": "v1.3.2",
    "model_version": "kde_v1.0.5",
    "confidence": 0.95
  },
  "audit_id": "audit_20250714_1032_analyst_103"
}
```

A.2 Network Pipeline API Contract

```
POST /api/network/graph
Authorization: Bearer <jwt>
Content-Type: application/json

{
  "seed_entity": {"type": "accused", "id": 1023},
  "depth": 2,
  "edge_types": ["co_accused", "financial_link", "mo_similarity"]
}
```



```
Response (200 OK):
{
  "result_id": "graph_20250714_1033",
  "nodes": [
    {"id": "accused:1023", "type": "accused", "label": "Ravi Kumar", "centrality": 0.87, "community": 1},
    {"id": "accused:1045", "type": "accused", "label": "Mohan Reddy", "centrality": 0.92, "community": 1}
  ],
  "edges": [
    {"source": "accused:1023", "target": "accused:1045", "type": "co_accused", "weight": 3.0, "firs": [452, 178, 892]},
    {"source": "accused:1023", "target": "account:9981", "type": "financial_link", "weight": 1.0, "amount": 450000}
  ],
  "communities": [
    {"id": 1, "size": 14, "label": "Cluster A (organized theft ring)", "avg_centrality": 0.74}
  ],
  "lineage": {
    "source_tables": ["fir_accused", "offender_history", "financial_transaction", "fir_mo"],
    "total_rows_touched": 1284,
    "algorithm": "louvain_v1.2",
    "cache_hit": true,
    "cache_key": "graph_accused_1023_d2_v3"
  },
  "audit_id": "audit_20250714_1033_analyst_103"
}
```

A.3 Predictive Pipeline API Contract

```
POST /api/predictive/forecast
Authorization: Bearer <jwt>
Content-Type: application/json
```

```
{
  "district": "bengaluru_north",
  "crime_type": "theft",
  "horizon_days": 30
}
```

```
Response (200 OK):
{
  "result_id": "forecast_20250714_1034",
  "forecast": [
    {"date": "2025-07-15", "predicted_count": 8.2, "lower_ci": 5.1, "upper_ci": 11.3},
    {"date": "2025-07-16", "predicted_count": 7.9, "lower_ci": 4.8, "upper_ci": 11.0}
  ],
  "confidence_interval_width": 6.2,
  "feature_importance": [
    {"feature": "unemployment_rate", "importance": 0.28, "direction": "positive"},
    {"feature": "recent_theft_count_7d", "importance": 0.22, "direction": "positive"},
    {"feature": "festival_period", "importance": 0.18, "direction": "positive"},
  ]
}
```

```
{
  "feature": "literacy_rate", "importance": 0.12, "direction": "negative"
},
"lineage": {
  "model_type": "gradient_boosted_regressor",
  "model_version": "automl_v2.3.1",
  "training_window": "2023-01-01 to 2025-06-30",
  "training_rows": 8760,
  "feature_pipeline_version": "v2.1.0",
  "cache_hit": true,
  "cache_ttl_hours": 24
},
"audit_id": "audit_20250714_1034_analyst_103"
}
```

A.4 Sample Synthetic FIR Data Row

```
-- Table: fir
-- Sample row for demo
INSERT INTO fir VALUES (
452, -- fir_id (PK)
'452/2025', -- fir_number
45, -- ps_id (FK -> Jayanagar PS)
12, -- crime_type_id (FK -> Theft IPC 379)
'2025-07-15', -- fir_date
'22:45:00', -- fir_time
'under_investigation', -- status
'Theft of laptop and cash from residence via forced entry through rear window.',
12.9279, -- lat (geocoded from address)
77.6271, -- lng
'Forced entry through rear window, ransacked bedroom, fled with laptop and Rs 15,000 cash'
);

-- Linked accused (repeat offender)
INSERT INTO fir_accused VALUES (452, 1023, 'primary', '2025-07-18',
'in_judicial_custody');
-- accused_id 1023 = Ravi Kumar, 4 prior offenses across 3 districts

-- Linked MO tag
INSERT INTO fir_mo VALUES (452, 17); -- mo_id 17 = "forced entry rear window"

-- Socio-economic context (district-level, joined via PS district)
-- district: bengaluru_urban, year: 2024
-- unemployment_rate: 6.2%, urbanization_rate: 89.1%, literacy_rate: 88.5%
```

These artifacts, combined with the schema in Section 5 and the pipeline specifications in Sections 7-9, give the development team everything needed to begin implementation on Day 1 of the sprint. The API contracts are ready to implement in Catalyst Functions, the lineage schema is ready to enforce in NoSQL, and the sample data shows the relational structure clearly. The blueprint is intentionally implementation-ready rather than aspirational: every design decision has been made, every trade-off has been resolved, and the only remaining work is execution.